

University of Piraeus
Department of Informatics

Assignment Report

From SHRDLU to Modern Grounded NLP

ShoeDLU: A Mini-SHRDLU-like System for Exploring Levels of Language Processing in a
Shoe-Rack World

In fulfilment of the requirements for the course
Generative AI, Large Language Models (LLM) and Prompt Engineering

Annemarie Bianka Weise
and
Jan Bossenz

Supervisor: Prof. Dr. Dimitris Panagoulas

Submission date: 2026-07-31

Contents

1	Introduction and Background	1
2	Methodology	1
2.1	Shared System Architecture and World Representation	1
2.2	Progression Across the Three Stages	2
2.3	Evaluation Methodology	3
3	Application and Implementation	4
3.1	World and Execution Layer	4
3.2	Parser Implementation	4
3.3	Grounding, Dialogue Management, and Stage-3 Integration	5
3.4	Training Data and Reproducibility	6
4	Discussion of Results	6
4.1	Component and Overall Performance	6
4.2	Effect of Stage 3	7
4.3	Limitations and Future Work	8
	Contribution Statement	I
	AI Usage Statement	I
	References	II
	Appendices	III

1 Introduction and Background

Artificial intelligence is playing an increasingly important role in modern work and production environments. In particular, the use of large language models (LLMs) increased strongly in recent years [5]. However, the basic idea of translating natural-language instructions into a form that machines can interpret and execute emerged long before modern LLMs. An early grounded natural-language understanding system that processed commands in a restricted microworld was SHRDLU. It was able to resolve linguistic references to objects in its blocks world, execute actions involving these objects, and update the world state. Object attributes, relations, and states could be used to identify objects and determine valid actions. Its architecture combined symbolic world representation, parsing, reference resolution, action planning, and world-state updates [6]. A reference is not only described by its attributes, but also by its relations in the current world. A description such as “the red object next to the box” defines constraints rather than a fixed identifier. A grounded command system must therefore distinguish language interpretation from reference resolution and detect whether no object, one object, or several objects satisfy a description [3].

Simple systems can achieve this through rules that search for predefined vocabulary or sentence structures associated with specific intents and relations. This first level of machine understanding is explicit, but also relatively limited in its handling of natural-language variation. Statistical classifiers and sequence models can reduce the dependence on manually defined rules by learning patterns from labelled examples. However, they remain strongly dependent on their training data, which are time-consuming to create and may contain biases or annotation errors [2]. Pretrained transformer models and LLMs provide broad capabilities across different NLP tasks [8]. This greater linguistic flexibility can also increase the risk of plausible but incorrect interpretations [1]. It is therefore important to assess in which parts of a grounded system such models can be useful and where explicit symbolic validation remains necessary.

This project investigates the gradual extension of a natural-language processing system. The examined microworld is a shoe-rack world in which shoes and tools can be described, placed, and used in different actions. The system is developed in three stages, progressing from rule-based processing to trained statistical models and finally to model-based correction. Stage 2 introduces logistic regression for intent classification and a conditional random field (CRF) for slot tagging. Stage 3 uses an open-source Hugging Face instruction model to check and correct selected parser outputs using information from the current world. All stages are evaluated in the same custom world, enabling a controlled comparison of the different language-understanding approaches.

2 Methodology

2.1 Shared System Architecture and World Representation

The system operates in a symbolic shoe-rack microworld containing shoes, cleaning and impregnation utensils, and repair tools. Shoes have descriptive attributes such as type, height, colour, and material, as well as mutable states for cleanliness, dirt, impregnation, dryness, and damage. Tools have a type, location, and a mutable fullness or

damage value. Objects can occupy three ordered shelves, the floor box, the drying area, the tool area, or the agent’s hand. The world represents the relations **on**, **inside**, **next_to**, and **holding**. Two objects on a shelf are **next_to** each other only when they occupy adjacent slots. The remaining areas are unordered, and objects in the same unordered area are treated as mutually adjacent. The hand contains at most one object. The system supports nine intents: **PICK_UP**, **PUT_DOWN**, **MOVE**, **CLEAN**, **DRY**, **IMPREGNATE**, **REPAIR**, **GO_ON_WALK**, and **GET_NEW_TOOL**.

All three stages share the same world model, grounding layer, dialogue handlers, and action rules. Only the parser component is exchanged. Each parser produces a compatible symbolic semantic frame containing an intent, action parameters, relations, and symbolic object references. These references do not yet contain resolved world-object identifiers. Instead, they contain an optional object class, attribute constraints, the original phrase, and relational constraints. Relations can be nested. For example, in “the boot next to the sneaker on the top shelf”, the boot is constrained by its relation to a sneaker, while the sneaker is additionally constrained by its shelf location. The system also distinguishes descriptive relations from target relations. A descriptive relation helps identify an object, whereas a target relation specifies the requested destination of an action. This distinction is particularly important for movement commands.

The processing pipeline begins with command normalisation and intent prediction. An optional Stage-3 check can validate or correct the intent. An intent-specific handler then determines which semantic roles must be grounded. The resolver evaluates each symbolic reference against the current world and returns **UNIQUE_MATCH**, **NO_MATCH**, **AMBIGUOUS**, or **INVALID_ID**, together with the relevant candidate set. The symbolic resolver never selects an arbitrary candidate. When Stage 3 is enabled, selected non-unique object or tool results may be passed to the model checker. When a Stage 3 proposal is accepted, the final resolution is changed to **UNIQUE_MATCH**, and the candidate set is replaced by the validated selected identifier. Rejected suggestions retain the original symbolic resolution result. Stage 3 may then propose one candidate, which must be validated against the current world and, in ambiguous cases, belong to the resolver’s candidate set. If the proposal is accepted, execution continues with the selected object. Otherwise, execution stops without changing the symbolic result. After successful grounding, the corresponding world method checks the action preconditions, executes the required steps, and updates the world state. It returns both an ordered action sequence and a description of the resulting changes. The complete architecture is shown in Fig. 1 in the Appendix.

2.2 Progression Across the Three Stages

Stage 1 uses the **RuleBasedParser**. It identifies intents and arguments through manually defined lexical patterns, structural rules, and alias mappings. The aliases convert linguistic variants into canonical world values. The results are deterministic and traceable, but their coverage is limited to anticipated vocabulary and sentence structures.

Stage 2 introduces two learned parser variants. The **IntentClassifierParser** replaces only rule-based intent detection. It uses TF-IDF unigrams and bigrams as features for a logistic regression, while argument extraction remains rule-based. The **SlotTaggerParser** combines the same intent classifier with a conditional random field.

The CRF assigns BIO labels to object attributes, semantic roles, relations, role boundaries, and action parameters. The predicted spans are then converted into the same symbolic reference structure as the rule-based output. Alias mappings still remain necessary because the resolver operates on the canonical world vocabulary.

Stage 3 adds an open-source Hugging Face instruction model as a constrained checking and correction component. It is not replacing the parser, executing actions, or overriding world constraints. It is used to validate predicted intents and assist with selected main-object, shoe, or tool references that do not produce a unique result. Target locations and `next_to` targets remain under symbolic resolution. A proposed intent must belong to the supported intent set and allow the selected parser to reparse the command. Proposed object identifiers must exist in the world. For an ambiguous result, the proposed identifier must also belong to the resolver’s candidate set. All corrections must exceed the configured confidence threshold. Invalid, low-confidence, or unparsable suggestions are rejected, and the original symbolic result is retained. This design separates linguistic correction from world validation and execution.

2.3 Evaluation Methodology

The evaluation is conducted at component and system level. At component level, the intent classifier is evaluated through accuracy, precision, recall, and F1 score. The CRF is evaluated through token-level precision, recall, and F1. The frequent non-semantic 0 label is excluded from the main weighted slot score.

The end-to-end evaluation contains 70 manually designed scenarios: 20 basic commands, 20 natural-language variations, 15 complex references, and 15 ambiguous or unresolvable references. Each scenario starts from a newly generated world with seed 205. Optional setup commands establish the state required for the evaluated command. A manually defined gold result specifies the expected intent, grounded arguments, resolution outcomes, execution status, and action result. The auxiliary `GET_NEW_TOOL` intent is excluded from this benchmark because it creates a new object from a specified class and type instead of grounding a reference to an existing object. The exact-match evaluation focuses on the main object, tool, target location, and `next_to` target. Secondary parameters such as walk length, place, weather, and repair target remain part of execution but are not scored separately. Five main metrics are reported:

- **Intent Accuracy** measures whether the final predicted intent matches the gold intent.
- **Grounded Frame Exact Match** requires the intent, main object, tool, target location, and `next_to` target to match.
- **Resolution Exact Match** requires the resolution state and candidate set of the object, tool, and target roles to match. Candidate order is ignored.
- **Execution Exact Match** requires both the execution status and the reported success value to match.
- **End-to-End Exact Match** requires the grounded frame, resolution results, and execution result to be correct simultaneously.

An unsuccessful action can therefore still be evaluated as correct when the command is interpreted and grounded correctly but violates a world constraint. Stage-3 interventions are also analysed separately. Accepted intent changes are classified as helpful or harmful, while accepted object and tool corrections are evaluated according to whether they select the expected identifier.

3 Application and Implementation

3.1 World and Execution Layer

The symbolic environment is implemented with the dataclasses `Shoe`, `CleaningUtensil`, `ImpregnationUtensil`, and `RepairTool`. The `World` class stores objects in type-specific dictionaries and represents shelves as ordered slot lists. Relations are derived from object locations and recalculated after each state change. Canonical values, aliases, placement restrictions, and action effects are centralised in `domain_rules_and_constants.py`.

The basic operations `pick_up` and `put_down` are reused by compound actions. For example, `move_object` first ensures that the requested object is held. It then places the object at a location or next to another object. The resulting message lists the individual execution steps. Domain-specific methods implement cleaning, drying, impregnation, repair, tool replacement, and walking. These actions update shoe states and tool resources, and may remove empty utensils or broken tools. The execution layer distinguishes recoverable preconditions from strict constraints. A recoverable precondition can be established through an intermediate step. For example, the agent can put down one object before picking up another. Strict constraints terminate the requested action. These include missing objects, incompatible tool types, full shelves, unavailable adjacent slots, and height-dependent placement restrictions.

Worlds can be defined manually or generated using `World.create_random`. Random generation respects the domain constraints, and a fixed seed ensures reproducibility. The evaluation uses seed 205 and produces a world containing 16 objects.

3.2 Parser Implementation

The abstract `BaseParser` class defines the shared parser interface. It normalises a command, determines or receives an intent, and calls the corresponding intent-specific composition method. For example, `MOVE` requires a main object and a target relation. `CLEAN` requires a shoe and a cleaning utensil. `GO_ON_WALK` requires a shoe, length, place, and weather. The resulting output uses the common symbolic reference format required by the resolver.

The `RuleBasedParser` detects intents through regular expressions and extracts canonical values using alias dictionaries. The matched action expression is removed before argument extraction to prevent action words, such as “dry”, from also being interpreted as object attributes. The `IntentClassifierParser` retains the rule-based argument extractor but replaces intent detection with a saved TF-IDF vectoriser and logistic regression classifier. The `SlotTaggerParser` combines the same intent classifier with a saved CRF model. The CRF assigns BIO labels

to object properties, semantic roles, relations, locations, tools, thresholds, and action parameters. Predicted values are then normalised through the shared alias mappings.

Commands with both a shoe and a tool require a role split. The rule-based parser splits cleaning, impregnation, and repair commands at expressions such as “with”, “using”, or “by using”. The phrase before the separator is assigned to the shoe role. The phrase after it is assigned to the tool role. The slot tagger performs the same conceptual split through predicted labels. The labels `BEFORE_SENTENCE_SPLIT` and `AFTER_SENTENCE_SPLIT` indicate the relative positions of the shoe and tool phrases. If no split label is predicted, the parser estimates the boundary from the predicted shoe and tool slots. The command is tagged once. The resulting spans are reused by the role-specific parsing methods.

All parser variants distinguish descriptive relations, which restrict an object reference, from target relations, which specify an action destination. The rule-based parser assigns these roles through intent-specific positional rules. For `MOVE`, the last detected relation is normally treated as the target, while earlier relations describe the object being moved. For `PUT_DOWN`, the first `next_to` relation is selected as the target; if none is present, the first detected relation is used. The slot tagger instead predicts explicit labels for descriptive and target relations. When a required target label is missing, it falls back to the same intent-specific rules. A predicted target location can also supply an `on` or `inside` target when the corresponding relation token was not labelled correctly.

Nested descriptive relations are converted into recursive object references. The current implementation supports these structures for descriptive references and `PUT_DOWN` targets. Nested descriptions of `MOVE` targets are not emitted because they are not yet consumed reliably by the dialogue layer. Each parser also retains the original object phrase so that the resolver can identify explicit world-object identifiers directly.

3.3 Grounding, Dialogue Management, and Stage-3 Integration

The resolver first checks a reference for an explicit identifier and then filters the available objects by class, canonical attributes, and recursive relational constraints. It returns a structured `ResolutionResult` containing the selected identifier, candidate identifiers, resolution state, and a diagnostic message. Target locations and `next_to` targets use a corresponding target-resolution structure.

After parsing and optional intent validation, the dialogue manager selects an intent-specific handler. The handler resolves the roles required by the action and invokes the corresponding world method. Shoe and tool references are resolved independently, while movement commands resolve the main object and target separately. `PUT_DOWN` additionally checks whether one of the matching objects is currently held. If the command contains no object description, the object id from the `holding` relation is used directly.

Stage 3 uses `Qwen/Qwen3-4B-Instruct-2507` [4, 7] through the Hugging Face inference client with temperature zero and constrained JSON output. This model was selected because it is an openly available instruction-tuned model

that can produce structured JSON responses. Its comparatively small parameter count also made remote inference more practical than using a substantially larger model. It is called for intent validation and selected non-unique object or tool references. Invalid responses, API failures, and rejected suggestions fall back to the symbolic result. Target locations and `next_to` targets are not corrected by the current implementation.

The final `InterpretationResult` stores the original and final intent, parsed frame, grounded identifiers, candidate sets, resolution states, execution status, action outcome, and Stage-3 correction flags. These values are used for both dialogue output and end-to-end evaluation.

3.4 Training Data and Reproducibility

The intent dataset contains 1,730 commands covering the nine supported intents. The same commands were converted into 16,836 token-level rows with BIO labels for the slot tagger. Both models use a stratified 75–25 train–test split with seed 404. The intent classifier uses TF–IDF unigram and bigram features, while the CRF uses token-level feature dictionaries. Initial commands and BIO annotations were produced with AI assistance and subsequently reviewed and corrected manually, particularly for relation attachment, role boundaries, and context-dependent labels. The final datasets and prompts are included in the project files.

4 Discussion of Results

4.1 Component and Overall Performance

The intent classifier achieved 95.8% accuracy. Its weakest class was `MOVE`, which was frequently confused with `PUT_DOWN` and `PICK_UP`, reflecting their shared action vocabulary (see Tables 1 and 2). The CRF reached a weighted token-level F1 score of 96.7% without the `O` label. Most attribute and tool labels performed strongly, while target-relation labels were less reliable, particularly `B-TARGET_NEXT_TO_RELATION` at 77.6% F1 (see Table 3). Errors in these labels can be fatal because they can change whether a relation describes an object or specifies the destination of an action.

At system level, both Stage-2 configurations achieved the strongest overall performance (see Table 4). The `IntentClassifierParser` and `SlotTaggerParser` both reached 100% Intent Accuracy and 91.4% End-to-End Exact Match. The rule-based system reached 92.9% Intent Accuracy and 84.3% End-to-End Exact Match. Learned intent detection therefore improved linguistic coverage, but the gap between component and end-to-end performance shows that correct intent prediction alone is insufficient. The complete task also depends on argument extraction, relation attachment, and ambiguity handling.

The category-level results reveal different strengths within the Stage 2 parsers (see Table 5). The intent-classifier configuration performed best on natural-language variation, with 90.0% End-to-End Exact Match, compared with 70.0% for the rule-based parser and 80.0% for the slot tagger. This indicates that learned intent detection improved the handling of alternative action formulations while the remaining rule-based argument extraction seems more

effective than the slot tagging. The slot tagger showed its main advantage on complex references, reaching 93.3%, compared with 73.3% for the intent-classifier configuration and 66.7% for the rule-based system. Its explicit labels for descriptive relations, target relations, and role boundaries therefore improved the interpretation of structurally complex commands. However, its lower score on natural-language variation suggests greater sensitivity to the formulations represented in the slot-training data.

The representative cases in Table 7 illustrate these differences. They show both a better paraphrase coverage of the learned intent classifier and the slot tagger’s advantage when descriptive and target relations must be separated. Overall, the results demonstrate that high component-level scores do not guarantee a correct grounded interpretation: a single intent, role-boundary, or relation error can change the resolved object, action target, and final execution result.

The gap between parser-level and end-to-end performance highlights the role of the world model. The parsers generate symbolic frames without consulting the current world, whereas grounding depends on the available objects, their attributes, locations, and spatial relations. For example, `next_to` refers to adjacent positions on ordered shelves but to all objects within the same unordered area. These modelling choices determine whether a reference resolves uniquely, remains ambiguous, or produces no match, while world constraints determine whether the resulting action can be executed. Stage 3 is also influenced by the world because its checks use the current world state.

4.2 Effect of Stage 3

Stage 3 produced some smaller improvements but reduced overall end-to-end performance. For the rule-based configuration, Intent Accuracy increased from 92.9% to 98.6% (see Table 4). All four accepted intent changes corrected an initially wrong prediction (see Table 6). These corrections improved End-to-End Exact Match from 70.0% to 95.0% for natural-language variations and from 66.7% to 80.0% for complex references (see Table 5). The learned parsers already achieved 100% Intent Accuracy, so Stage 3 provided no additional intent corrections for these configurations.

Despite the intent improvements, enabling Stage 3 reduced overall End-to-End Exact Match (see Table 4). From 84.3% to 78.6% for the rule-based system, from 91.4% to 81.4% for the intent-classifier system, and from 91.4% to 82.9% for the slot-tagger system. The decrease was caused by wrong object corrections, with only about one quarter of the evaluated changes selecting the expected object (see Table 6). Consequently, Grounded Frame and Resolution Exact Match also decreased for Stage 3. This negative effect mainly occurred for ambiguous or unresolvable references. Without Stage 3, all three parser configurations achieved 100% End-to-End Exact Match in this category because they preserved ambiguous candidate sets or correctly returned no match. With Stage 3, the performance fell drastically for every configuration (see Table 5). As illustrated by the ambiguity example in Table 7, the model often selected a candidate even though the command did not contain enough information to identify one object uniquely. Its confidence therefore represented a preferred interpretation rather than evidence that the reference was unambiguous.

These results suggest that Stage 3 is most useful for correcting unsupported language when the intended interpretation is still uniquely supported by the world. It is less suitable for genuine ambiguity. A safer integration should distinguish `NO_MATCH` from `AMBIGUOUS`: constrained model-based recovery could remain available for no-match cases, whereas ambiguous references should normally only trigger a clarification request.

4.3 Limitations and Future Work

The results are limited by the restricted domain, training data, and evaluation design. Although the datasets contain substantial variation within the shoe-rack world, they cannot cover all possible command formulations. This is particularly relevant for the slot tagger, whose BIO annotations required extensive manual correction. Remaining inconsistencies in relation attachment, role boundaries, and target labels may therefore have influenced its performance. The evaluation also uses a limited set of manually designed scenarios and does not establish generalisation across different object arrangements. Future experiments should include multiple world seeds, additional paraphrases, and more structurally ambiguous commands.

A central limitation is that relation attachment still depends partly on fixed positional rules. In some commands, a relation can plausibly describe either the main object or the action target. A more robust system could generate several candidate interpretations and validate them against the current world. If more than one interpretation remains valid, the dialogue manager should request clarification rather than select one automatically. This is consistent with the Stage-3 results: model-based correction can assist with unfamiliar but uniquely resolvable language, but it should not replace explicit ambiguity handling.

Stage 3 is currently limited to intent validation and selected object and tool references. Target locations and `next_to` targets remain entirely symbolic. These roles could be added while retaining symbolic validation, but recovery should preferably be restricted to `NO_MATCH` cases as well. The system also lacks sufficient interaction history to resolve expressions such as “a replacement for the broken one” or “another tool like the previous one”. Supporting such references would require an action history or access to previous world states.

Finally, the evaluation does not separately score secondary parameters such as walk length, place, weather, and repair target. Because missing values are replaced with deterministic defaults, an end-to-end result may be correct even when one of these parameters was not extracted. Future evaluations should include these fields in the grounded frame. Overall, the findings support a hybrid architecture in which learned models improve linguistic coverage while symbolic components preserve grounding, constraint checking, and ambiguity.

Contribution Statement

The project was developed collaboratively by Annemarie Bianka Weise and Jan Bossenz. The overall system architecture, core design decisions, domain definitions, evaluation methodology, and interpretation of the results were discussed and agreed upon jointly.

Annemarie was primarily responsible for implementing the symbolic world structure, the rule-based parser, and most parts of the dialogue manager. Jan was primarily responsible for implementing the model training scripts, the intent-classifier parser and the CRF-based slot-tagger parser. The resolver combines contributions from both authors. The shared `BaseParser` architecture was developed jointly through several revisions.

Both authors reviewed and corrected the training datasets together. The evaluation notebooks, test scenarios, metric definitions, analysis of the results, and final report were also developed collaboratively. All components were reviewed and discussed by both authors, and both share responsibility for the submitted system and report.

AI Usage Statement

Generative AI tools were used as supporting tools during data preparation, software development, and report revision. As described in the report, AI assistance was used to generate initial versions of the intent-classification and slot-labelling datasets. All generated examples and annotations were subsequently reviewed manually. Invalid or unnatural commands were removed, incorrect labels were corrected, and additional examples and vocabulary were added by the authors.

During software development, AI tools were occasionally used to support debugging and to improve code readability and formatting. These changes concerned presentation, structure, and error identification rather than the independent generation of complete system components. No fully AI-generated implementation was integrated into the project. The initial code and the fundamental algorithms, architecture, and design decisions were developed by the authors, while AI suggestions were reviewed and, where appropriate, manually adapted.

AI assistance was also used to check the report for grammatical and spelling errors and to shorten or clarify selected passages. This was necessary because the initial report substantially exceeded the intended length. The technical content, experimental results, interpretations, and final wording were reviewed and approved by both authors.

References

- [1] Lei Huang et al. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *ACM Trans. Inf. Syst.* 43.(2) (2025). DOI: 10.1145/3703155.
- [2] Jan-Christoph Klie, Bonnie Webber, and Iryna Gurevych. “Annotation Error Detection: Analyzing the Past and Present for a More Coherent Future”. In: *Computational Linguistics* 49.(1) (2023), pp. 157–198. DOI: 10.1162/coli_a_00464.
- [3] Cynthia Matuszek. “Grounded Language Learning: Where Robotics and NLP Meet”. In: *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*. 2018, pp. 5687–5691. DOI: 10.24963/ijcai.2018/810.
- [4] Qwen Team. *Qwen3-4B-Instruct-2507*. Hugging Face model card. Last Accessed: 2026-07-29. 2025. URL: <https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>.
- [5] Stanford Institute for Human-Centered Artificial Intelligence. *The 2026 AI Index Report*. Tech. rep. Stanford University, 2026. URL: https://hai.stanford.edu/assets/files/ai_index_report_2026.pdf.
- [6] Terry Winograd. “Understanding Natural Language”. In: *Cognitive Psychology* 3.(1) (1972), pp. 1–191. DOI: 10.1016/0010-0285(72)90002-3.
- [7] An Yang et al. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: <https://arxiv.org/abs/2505.09388>.
- [8] Wayne Xin Zhao et al. *A Survey of Large Language Models*. 2026. URL: <https://arxiv.org/abs/2303.18223>.

Appendices

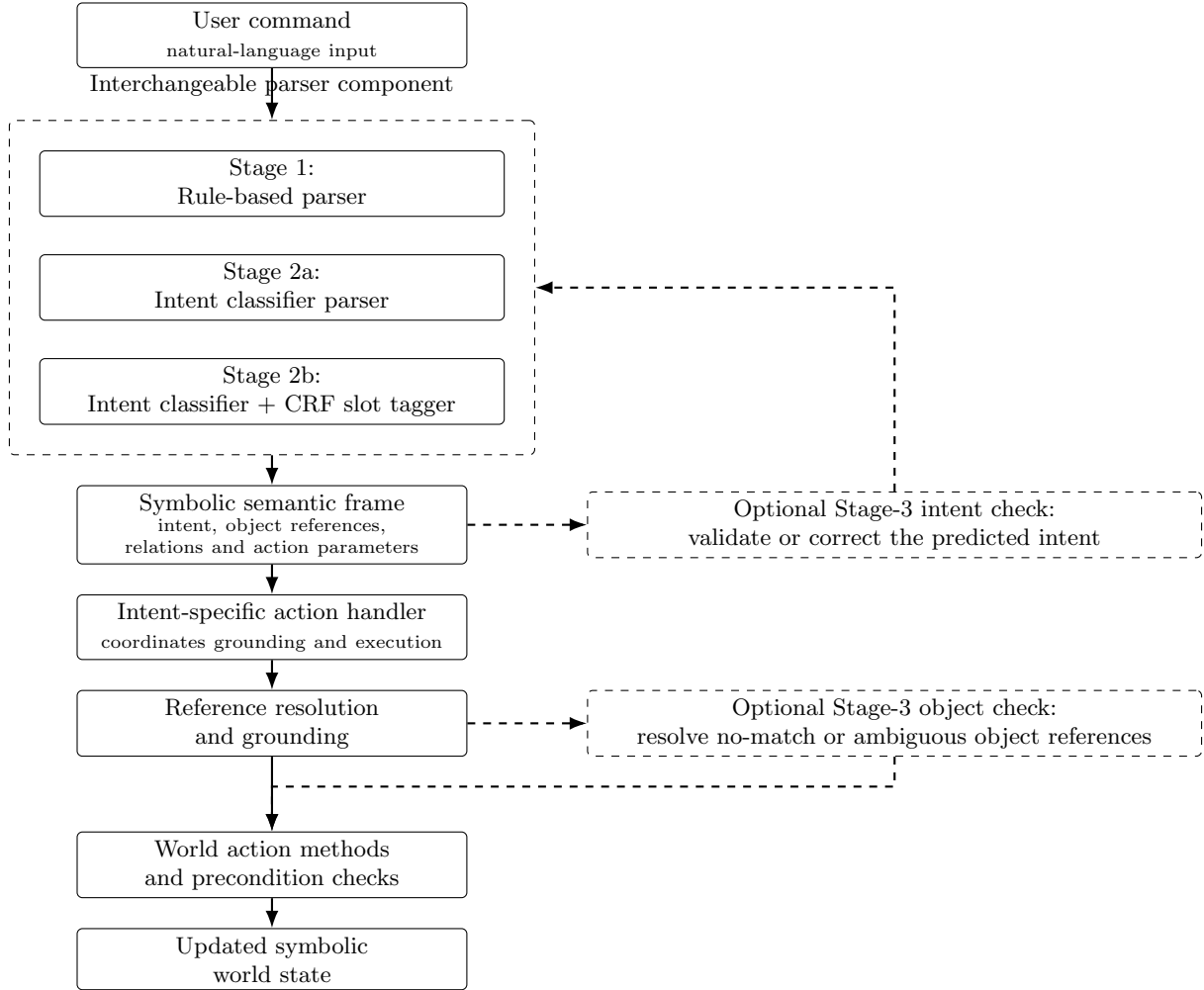


Figure 1: The shared grounded language-processing pipeline of this work. The exchangeable parser implementations produce symbolic semantic frames with the same structure. Before the intent-specific handler is selected, the Stage-3 model checker can validate or correct the predicted intent and rerun the parser with this intent. The handler then coordinates reference resolution and can perform additional Stage-3 checks for unresolved or ambiguous object references before calling the symbolic world action.

Table 1: Class-level results of the intent classifier on the random test split. The overall accuracy was 95.8%.

Intent	Precision	Recall	F1	Support
CLEAN	100.0	98.0	99.0	54
DRY	100.0	91.0	95.0	44
GET_NEW_TOOL	97.0	95.0	96.0	40
GO_ON_WALK	98.0	100.0	99.0	45
IMPREGNATE	95.0	100.0	97.0	56
MOVE	92.0	88.0	90.0	51
PICK_UP	96.0	90.0	93.0	49
PUT_DOWN	89.0	100.0	94.0	42
REPAIR	96.0	100.0	98.0	52
Macro average	96.0	96.0	96.0	433
Weighted average	96.0	96.0	96.0	433

Table 2: Confusion matrix of the intent classifier on the random test split. Rows show the true intent and columns show the predicted intent.

True intent	CLEAN	DRY	GET_NEW_TOOL	GO_ON_WALK	IMPREGNATE	MOVE	PICK_UP	PUT_DOWN	REPAIR
CLEAN	53	0	0	0	0	0	1	0	0
DRY	0	40	0	1	2	0	1	0	0
GET_NEW_TOOL	0	0	38	0	1	0	0	0	1
GO_ON_WALK	0	0	0	45	0	0	0	0	0
IMPREGNATE	0	0	0	0	56	0	0	0	0
MOVE	0	0	0	0	0	45	0	5	1
PICK_UP	0	0	1	0	0	4	44	0	0
PUT_DOWN	0	0	0	0	0	0	0	42	0
REPAIR	0	0	0	0	0	0	0	0	52

Table 3: Token-level results of the CRF slot tagger on the random test split. The non-semantic 0 label is excluded. The overall weighted F1 score was 96.7%.

Label	Precision	Recall	F1	Support
B-AFTER_SENTENCE_SPLIT	89.7	78.8	83.9	33
B-BEFORE_SENTENCE_SPLIT	100.0	97.9	98.9	95
B-CLEANING_STATUS	87.0	100.0	93.0	20
B-CLEANING_TOOL_TYPE	100.0	100.0	100.0	63
B-COLOR	99.6	100.0	99.8	224
B-DIRT_TYPE	100.0	100.0	100.0	10
B-DRY_STATUS	100.0	88.9	94.1	9

Continued on next page

Table 3 – continued from previous page

Label	Precision	Recall	F1	Support
B-IMPREGNATION_STATUS	100.0	100.0	100.0	10
B-IMPREGNATION_TOOL_TYPE	100.0	100.0	100.0	54
B-LOCATION_RELATION	93.9	85.3	89.4	109
B-MATERIAL	100.0	100.0	100.0	88
B-MATERIAL_STATUS	100.0	100.0	100.0	10
B-NEXT_TO_RELATION	94.9	95.8	95.3	96
B-OBJECT_ID	100.0	100.0	100.0	201
B-OBJECT_LOCATION	99.0	94.4	96.7	107
B-REPAIR_TARGET	100.0	100.0	100.0	25
B-REPAIR_TOOL_TYPE	98.2	98.2	98.2	57
B-SHOE_HEIGHT	100.0	100.0	100.0	10
B-SHOE_TYPE	99.3	99.3	99.3	290
B-SOLE_STATUS	85.7	100.0	92.3	6
B-TARGET_LOCATION	86.5	98.5	92.1	65
B-TARGET_LOCATION_RELATION	77.8	95.5	85.7	66
B-TARGET_NEXT_TO_RELATION	79.2	76.0	77.6	25
B-THRESHOLD	100.0	100.0	100.0	33
B-WALK_LENGTH	100.0	93.3	96.6	15
B-WALK_PLACE	100.0	100.0	100.0	20
B-WALK_WEATHER	100.0	100.0	100.0	10
I-CLEANING_STATUS	100.0	87.5	93.3	8
I-IMPREGNATION_STATUS	100.0	100.0	100.0	1
I-NEXT_TO_RELATION	94.6	95.6	95.1	91
I-OBJECT_LOCATION	100.0	93.8	96.8	81
I-REPAIR_TOOL_TYPE	100.0	100.0	100.0	57
I-TARGET_LOCATION	86.4	100.0	92.7	57
I-TARGET_NEXT_TO_RELATION	79.2	79.2	79.2	24
I-THRESHOLD	100.0	100.0	100.0	14
I-WALK_PLACE	100.0	100.0	100.0	1
I-WALK_WEATHER	100.0	100.0	100.0	8
Micro average	96.4	96.9	96.7	2093
Macro average	96.0	96.2	96.0	2093
Weighted average	96.7	96.9	96.7	2093

Table 4: Overall pipeline results across all 70 evaluation scenarios. All values are percentages. EM denotes Exact Match.

System	Intent Accuracy	Grounded Frame EM	Resolution EM	Execution EM	End-to-End EM
Rule-based	92.9	84.3	87.1	88.6	84.3
Rule-based + Stage 3	98.6	78.6	78.6	85.7	78.6
Intent classifier	100.0	91.4	91.4	92.9	91.4
Intent classifier + Stage 3	100.0	81.4	81.4	88.6	81.4
Slot tagger	100.0	91.4	91.4	92.9	91.4
Slot tagger + Stage 3	100.0	82.9	82.9	87.1	82.9

Table 5: End-to-End Exact Match by evaluation category. The categories contain 20 basic, 20 natural-language, 15 complex-reference, and 15 ambiguous or unresolvable scenarios. All values are percentages.

System	Basic	Natural-language variation	Complex reference	Ambiguous or unresolvable
Rule-based	100.0	70.0	66.7	100.0
Rule-based + Stage 3	100.0	95.0	80.0	26.7
Intent classifier	100.0	90.0	73.3	100.0
Intent classifier + Stage 3	100.0	100.0	80.0	33.3
Slot tagger	95.0	80.0	93.3	100.0
Slot tagger + Stage 3	100.0	90.0	93.3	40.0

Table 6: Summary of accepted Stage-3 corrections. Object and tool results are shown as correct changes divided by all evaluated changes of the corresponding type.

System	Intent changes	Helpful intent changes	Correct object changes	Correct tool changes
Rule-based + Stage 3	4	4	3/13	2/3
Intent classifier + Stage 3	0	0	3/12	2/3
Slot tagger + Stage 3	0	0	3/11	1/2

Table 7: Exemplary qualitative results from the end-to-end evaluation. RB denotes “rule-based” parser, IC “intent-classifier” parser, ST “slot-tagger” parser, and S3 “Stage-3” model checker usage.

Case	Command	Observed results	Main finding
Natural variation (natural_05)	<i>“Take the wet boot off the top shelf and put it into the drying area.”</i>	• RB predicted PICK_UP and returned NO_MATCH • IC and ST correctly predicted MOVE, resolved boot_1, and selected drying_area • RB+S3 corrected the intent and executed the command successfully	Learned intent detection and Stage3 correction can handle an indirect movement phrasing.
Natural paraphrase (natural_12)	<i>“Could you remove the grass from the slightly dirty white rubber trainer with brush_1?”</i>	• RB did not detect a intent • RB+S3 produced no correction • IC and ST correctly predicted CLEAN, resolved sneaker_2 and brush_1, and executed the action	The trained intent model covers paraphrases that are absent from the rule vocabulary.
Complex reference (complex_ref_05)	<i>“Move the low wet canvas boot beside the clean blue leather sneaker to the middle shelf.”</i>	• RB and IC resolved boot_1, but used sneaker_1 as movement target • ST identified middle_shelf as the target • S3 had no effect here	Explicit slot labels can improve the correct identification of target relations.
Genuine ambiguity (truly_ambiguous_02)	<i>“Pick up the white shoe.”</i>	• RB, IC, and ST correctly returned AMBIGUOUS with candidates {sneaker_2, boot_2} • All S3 configurations incorrectly selected sneaker_2 and executed the command	S3 can replace valid ambiguity with an unsupported but plausible choice.